

# MAC Automation Suite

## End-Stage Diagram and Technical Documentation

---

LeCroy MAUI acquisition workflow  
WF? ALL parser integration  
Artifact generation and post-processing flow

### Purpose

This package explains how the automation suite executes a run from configuration load through waveform acquisition, binary parsing, storage, prettification, optional post-processing, and Brain-controlled sweep looping.

### Intended readers

Researchers and developers maintaining the MATLAB-based MAC automation suite for Teledyne LeCroy oscilloscopes.

*Generated by ChatGPT based on the current automation-suite design, the uploaded parser file, and the earlier suite structure discussed in this conversation.*

# End-Stage Runtime Diagram

# MAC Automation Suite — End-Stage Runtime Flow

Configuration → Acquisition → WF? ALL Parse → Storage → Prettify → Optional Post-Processing → Brain Loop Control



Implementation note: the parser expects the actual visadev object (session.io), not the wrapper session.

The end-stage flow shows the sequence after a run is requested: configuration, scope session creation, single-shot acquisition, WF? ALL parsing, artifact storage, prettification, optional post-processing, and Brain loop control.

# 1. Architecture Overview

The automation suite is organized as a layered pipeline. A Brain controller decides when a run begins and when a sweep ends. The acquisition layer opens the LeCroy connection and coordinates a single-shot measurement. The parser layer uses WF? ALL to read a complete binary waveform block, decodes WAVEDESC metadata plus DAT1 samples, and returns volts and time arrays for each channel. The storage layer preserves both machine-oriented and human-oriented artifacts, and the optional post-processing layer consumes the same CSV shape as the legacy workflow.

## Major responsibilities

- Configuration defines the behavior of every layer: connection, acquisition, transfer, storage, logging, Brain control, and post-processing.
- `acquireRun` executes one end-to-end run and writes run-folder artifacts.
- `lecroy_waveform_all_parser` performs transport setup, binary block reading, and waveform decoding.
- `prettifyRunArtifacts` creates cleaner companion files that are easier to inspect than the raw MAT/JSON outputs.
- The post-processing package remains logically separate so acquisition and analysis can evolve independently.

# 2. Core Function Responsibilities

## `defaultConfig()`

`defaultConfig` produces the master configuration struct consumed by the rest of the suite. It is the single source of truth for the runtime defaults.

- `connection`: VISA resource string, terminators, timeouts, visadev preference, and clear-on-connect behavior.
- `acquisition`: channel list, channel labels, run index, setup commands, arm mode, wait timers, and pre/post-arm behavior.
- `transfer`: parser selection, parser options, and WF? ALL transport settings.
- `storage`: root directory, run folder pattern, CSV header lines, MAT/JSON writes, and prettified companion artifact toggles.
- `brain`: sweep duration, inter-run pause, error policy, and whether processing runs automatically after acquisition.
- `postprocess`: settings passed to the downstream MAC analysis functions.

## Brain / `runSweep`

The Brain layer orchestrates repeated runs over a fixed wall-clock interval. Conceptually, it is a scheduler around `acquireRun` rather than a replacement for it.

- Starts the first run immediately or on the next scheduled tick.
- Stops after `runDurationSeconds` elapses.

- Sleeps for `pauseBetweenRunsSeconds` between runs.
- Optionally launches post-processing after acquisition completes.
- Stops or continues on errors depending on `stopOnError`.

## acquireRun(cfg)

`acquireRun` is the main run executor. It opens the instrument session, prepares the scope, arms the acquisition, loops over requested channels, calls the parser, and writes artifacts.

- Calls `lecroy.connect(cfg)` to obtain a session object.
- Uses `session.io` for low-level reads and writes. This matters because the parser expects the actual `visadev` object.
- Executes STOP, CLSW, and user-defined `setupCommands` when enabled.
- Arms TRMD SINGLE and waits for completion using `*OPC?`.
- Reads each requested channel exclusively through the `WF? ALL` parser.
- Writes per-channel CSV files, `acquisition.mat`, `manifest.json`, and prettified companion outputs.
- Returns a result struct with paths and parsed data.

## lecroy.connect(cfg)

The connection layer is responsible for building the communication session wrapper. In the current suite shape, the wrapper exposes `session.io` as the `visadev` object used by transport functions.

- Resolves the VISA resource string.
- Applies terminators and session timeout defaults.
- Optionally clears stale I/O state on connect.
- Normalizes the connection into a structure the rest of the suite can consume.

## lecroy\_waveform\_all\_parser(io, chan, ...)

The parser is a self-contained file that sets up the transfer, reads a complete IEEE-488.2 arbitrary block, and converts the returned data into MATLAB-friendly arrays and metadata.

- Uses CHDR SHORT, CFMT DEF9,WORD,BIN, CORD LO, and WFSU SP,1,NP,0,FP,0,SN,0 by default.
- Optionally triggers a single-shot acquisition if `AcquireSingle` is enabled.
- Sends `Cn:WF? ALL` and reads the entire binary block without relying on line-oriented reads.
- Finds the WAVEDESC anchor in the payload and decodes descriptor fields at fixed offsets.
- Reads DAT1, converts raw sample codes into volts using vertical gain and vertical offset, and computes a time axis from descriptor fields or optional INSPECT? fallback values.
- Returns a struct containing raw samples, scaled volts, block metadata, descriptor metadata, and recommended timeout information.

## prettifyRunArtifacts(runDir, result, cfg)

The prettify stage does not replace the raw artifacts. Instead, it generates companion files that summarize the run in a cleaner form.

- pretty\_manifest.json reorganizes and normalizes the stored manifest.
- acquisition\_summary.json captures the practical run summary and per-channel numeric statistics.
- acquisition\_summary.txt provides a plain-language human-readable report for quick review.
- An optional acquisition\_pretty.mat can store a lighter-weight summary MAT file.

### 3. WF? ALL Parser: Internal Mechanics

#### Transport setup

Before any channel read, the parser configures a deterministic transfer format. The settings are chosen to make the host-side binary decode stable on typical Intel MATLAB systems.

- CHDR SHORT keeps the reply prefix compact.
- CFMT DEF9,WORD,BIN requests binary 16-bit waveform samples.
- CORD LO requests little-endian byte ordering.
- WFSU SP,1,NP,0,FP,0,SN,0 requests a full waveform without sparsing.

#### Binary block read

WF? ALL returns an IEEE-488.2 arbitrary block. The parser reads it in a length-driven way rather than line-by-line.

- Scan incoming bytes until the # character that marks the start of the arbitrary block header.
- Read the digit count immediately after #.
- Read the decimal payload length encoded by that digit count.
- Read exactly that many payload bytes.
- Optionally consume one trailing terminator byte if present.

#### Descriptor decode

Inside the payload, the parser locates the literal string WAVEDESC and treats it as the zero-point for the descriptor offsets.

- COMM\_ORDER is read first to establish the byte order used for the rest of the descriptor.
- Core sizes are then decoded: waveDescLen, userTextLen, trigtimeLen, risTimeLen, waveArray1Len, and waveArray2Len.
- Useful metadata such as instrumentName, traceLabel, waveArrayCount, verticalGain, verticalOffset, horizInterval, and horizOffset are extracted next.
- The parser walks forward through the payload by descriptor-reported segment lengths until it reaches DAT1.
- DAT1 is interpreted as int8 or int16 depending on COMM\_TYPE.
- Scaled volts are computed as  $\text{raw} * \text{verticalGain} - \text{verticalOffset}$ .
- The time axis is derived from  $\text{horizOffset} + \text{index} * \text{horizInterval}$  when available.

## Timeout strategy

The parser can accept a fixed TimeoutSeconds value, but it also returns a recommended timeout estimate based on actual payload size. This allows the suite to adjust future runs for long captures.

## 4. Artifact Set Written Per Run

| Artifact                 | Purpose                                                  | Primary Consumer                  |
|--------------------------|----------------------------------------------------------|-----------------------------------|
| scope_<run>_<ch>.csv     | Legacy-compatible two-column time and amplitude export.  | Post-processing pipeline          |
| acquisition.mat          | Machine-oriented archive of channelData and manifest.    | Developers and recovery workflows |
| manifest.json            | Structured description of configuration and run context. | Debugging and provenance          |
| pretty_manifest.json     | Normalized manifest companion with channel summary.      | Human inspection                  |
| acquisition_summary.json | Compact run summary with per-channel statistics.         | Dashboards and quick checks       |
| acquisition_summary.txt  | Readable plain-text summary of the run.                  | Operators and reviewers           |

## 5. Configuration Guide

The most important runtime settings are summarized below. In practice, these should be edited in defaultConfig and overridden per experiment only when necessary.

| Config path                       | What it controls                        | Typical value / note                                            |
|-----------------------------------|-----------------------------------------|-----------------------------------------------------------------|
| cfg.connection.resource           | Visa resource string                    | USB0::...::INSTR                                                |
| cfg.connection.timeoutSeconds     | Session-level default timeout           | Increase for long WF? ALL transfers                             |
| cfg.acquisition.channels          | Channels included in each run           | [1 2 3 4]                                                       |
| cfg.acquisition.setupCommands     | Scope commands applied before arming    | VDIV, OFST, trigger, timebase                                   |
| cfg.acquisition.acquireMode       | Run arm mode                            | "single"                                                        |
| cfg.transfer.mode                 | Readback path selector                  | "WFALL"                                                         |
| cfg.transfer.parser.functionName  | Parser function to call                 | lecroy.lecroy_waveform_all_parser or lecroy_waveform_all_parser |
| cfg.transfer.parser.commFormat    | Binary transfer format                  | "DEF9,WORD,BIN"                                                 |
| cfg.transfer.parser.waveformSetup | WF? ALL point-selection settings        | "SP,1,NP,0,FP,0,SN,0"                                           |
| cfg.storage.rootDir               | Top-level output folder                 | Experiment data directory                                       |
| cfg.storage.writePrettyArtifacts  | Enable human-readable companion outputs | true                                                            |
| cfg.brain.runDurationSeconds      | Sweep wall-clock duration               | Operator-chosen                                                 |
| cfg.brain.processAfterAcquire     | Run MAC post-processing automatically   | true or false                                                   |



## Integration note

If `acquireRun` obtains a session wrapper rather than a bare `visadev` object, the parser must receive `session.io`. Likewise, the parser function name in the config must match the file location on the MATLAB path. If the parser file is moved into `+lecroy/`, the correct `functionName` becomes `lecroy.lecroy_waveform_all_parser`.

## 6. End-to-End Run Sequence

1. Brain decides a run should start.
2. `defaultConfig` (or an experiment-specific override) is loaded.
3. `lecroy.connect(cfg)` opens the communication session.
4. `STOP` / `CLSW` / `setupCommands` prepare the instrument.
5. `TRMD SINGLE` arms the scope, and `*OPC?` waits for completion.
6. For each requested channel, `acquireRun` calls the `WF? ALL` parser.
7. The parser configures transfer formatting, sends `Cn:WF? ALL`, and reads the full binary block.
8. The parser decodes `WAVEDESC` and `DAT1`, then returns time, amplitude, raw samples, and metadata.
9. `acquireRun` writes CSV, MAT, and JSON outputs into the run folder.
10. `prettyRunArtifacts` generates operator-friendly summaries.
11. If enabled, `postprocess.process_run` executes on the completed run folder.
12. Brain decides whether to pause and loop or end the sweep.

## 7. Common Failure Modes and Checks

- Parser function not found: verify the MATLAB path and the package-qualified `functionName` string in `cfg.transfer.parser.functionName`.
- Wrong object passed to the parser: pass `session.io`, not the wrapper session, when the parser expects a `visadev` object.
- Timeouts during `WF? ALL`: increase `cfg.connection.timeoutSeconds` and `cfg.transfer.parser.timeoutSeconds`, or reduce waveform size.
- Inconsistent reads caused by readline on `WF? ALL`: always use the binary block reader, never a line reader, for waveform blocks.
- Unexpected scaling: verify `VDIV`, `OFST`, `COMM_FORMAT`, `COMM_ORDER`, and the descriptor-derived vertical gain and vertical offset.
- Post-processing mismatch: confirm that the CSV files remain two-column exports with time in column one and amplitude in column two.

## 8. Recommended Maintenance Practices

- Keep `defaultConfig` conservative and stable. Experimental overrides should live outside the shared baseline.

- Treat the parser as a protocol module. Avoid mixing line-based query helpers into waveform block reads.
- Preserve the raw machine files and write human-readable companion files instead of mutating the originals.
- Review one completed run folder after every major acquisition or parser change before launching a long Brain-controlled sweep.
- When long-memory captures are enabled, validate timeout settings and descriptor fields on a short test run first.